

Planck Automation

Determinação da constante de Planck por radiação de corpo negro

Manual Técnico

Arquitetura, modelo físico e lógica interna

6 de agosto de 2026

Resumo

Este documento descreve como o software funciona por dentro: a arquitetura em camadas, o modelo de concorrência, a cadeia de cálculo que converte medidas elétricas na constante de Planck, os drivers dos instrumentos e a bancada simulada. Destina-se a quem vai ler, manter ou estender o código. Para operar o software, consulte o **Manual do Usuário**.

Sumário

1	Arquitetura	2
1.1	Camadas	2
1.2	Mapa de arquivos	2
1.3	Ciclo de vida	2
2	Configuração persistente	3
3	Camada de hardware	3
3.1	O driver da fonte	3
3.2	O driver do multímetro	4
3.3	Threads de infraestrutura	4
3.4	Seleção de drivers e modo demonstração	4
4	O modelo físico	4
4.1	Da resistência à temperatura	5
4.2	Da temperatura à constante de Planck	6
4.3	A regressão	6
4.4	O limiar de confiança e a região de Wien	6
4.5	A simulação analítica	7
5	Teoria de erros	8
5.1	Vocabulário (GUM)	8
5.2	Aleatório versus sistemático: a distinção que organiza tudo	9
5.3	A cadeia completa	9
5.4	Como ler o χ^2 reduzido neste experimento	10
5.5	O orçamento de incertezas	11
5.6	O que NÃO está no orçamento	11

6	A bancada simulada	11
6.1	Por que os dois drivers compartilham estado	11
6.2	O modelo térmico	11
6.3	Como as constantes foram obtidas	12
6.4	Erros e ruídos reproduzidos	12
6.5	O que o mock ainda não reproduz	12
7	Concorrência	13
7.1	Por que threads	13
7.2	Os sinais	13
7.3	Parada segura	13
8	O fluxo de um experimento	13
9	Formato dos dados	14
9.1	O CSV de coleta	14
9.2	O relatório PDF	15
10	Perfis e rastreabilidade	15
10.1	O sistema de perfis	15
10.2	O painel de parâmetros compartilhado	15
10.3	Metadados por coleta	16
11	Camada de conteúdo	16
12	A interface Fluent	16
12.1	O que mudou e por quê	16
12.2	Estrutura	17
12.3	Configuração que saiu do código	17
12.4	Transição	17
13	Aparência	17
14	Como estender	18
15	Testes	18

1 Arquitetura

1.1 Camadas

O software se organiza em quatro camadas, com dependências apontando sempre para baixo:

Camada	Responsabilidade e conteúdo
ui/	Tudo que o usuário vê: janela principal, abas, painéis, tema. Não conhece VISA nem faz contas de física.
core/	Comunicação com os instrumentos: drivers SCPI, varredura e validação de recursos, bancada simulada.
utils/	Funções puras: o modelo físico-matemático e a geração do relatório PDF. Não dependem de Qt nem de hardware.
content/	Dados tipados que não são código — hoje, a lista de referências bibliográficas.

1.2 Mapa de arquivos

Arquivo	Linhas	Papel
main.py	18	Ponto de entrada: cria o <code>QApplication</code> , aplica o estilo Fusion e o tema escuro, abre a janela.
ui/main_window.py	59	<code>MainWindow</code> : monta as quatro abas e garante o desligamento seguro ao fechar.
ui/theme.py	119	Folha de estilos QSS do tema escuro.
ui/components/connection_panel.py	171	Painel de ligações: varredura, validação, modo demonstração e persistência das preferências.
ui/components/export_dialog.py	39	Janela de metadados do relatório.
ui/tabs/tab_simulation.py	325	<code>SimulationWorker</code> e a interface da simulação analítica.
ui/tabs/tab_experiment.py	445	<code>ExperimentWorker</code> e a interface da coleta na bancada.
ui/tabs/tab_references.py	127	Renderiza as fichas de referência.
core/hardware_manager.py	257	Drivers SCPI reais, threads de varredura e validação, seleção de drivers.
core/mock_hardware.py	300	Bancada simulada e drivers falsos.
utils/math_models.py	86	O modelo físico: temperatura, simulação e regressão.
utils/pdf_exporter.py	79	Montagem do relatório PDF com reportlab.
content/referencias.py	145	Lista tipada das referências.

1.3 Ciclo de vida

main.py faz três coisas e sai do caminho:

```

app = QApplication(sys.argv)
app.setStyle("Fusion")           # motor de renderização neutro
app.setStyleSheet(DARK_THEME)   # QSS do tema escuro
window = MainWindow()
window.show()
sys.exit(app.exec())

```

`MainWindow` instancia um único `HardwareManager` e o compartilha entre o painel de ligações e a aba de experimento. Ao fechar a janela, o `closeEvent` verifica se há coleta em curso; se houver, sinaliza a parada e *aguarda* a thread terminar, para que a fonte seja desligada antes de o processo morrer.

2 Configuração persistente

O software guarda preferências no registro do Windows via `QSettings`, sob a organização `Senac` e a aplicação `PlanckAutomation`. As chaves usadas:

Chave	Conteúdo
<code>Connection/LastPWSName</code>	Nome amigável da última fonte validada
<code>Connection/LastPWSRes</code>	Endereço VISA da última fonte validada
<code>Connection/LastDMMName</code>	Nome amigável do último multímetro validado
<code>Connection/LastDMMRes</code>	Endereço VISA do último multímetro validado
<code>Connection/DemoMode</code>	Booleano: modo demonstração ligado

Os nomes da organização e da aplicação estão centralizados em `core/hardware_manager.py` (constantes `ORGANIZACAO` e `APLICACAO`), acessíveis pela função `preferencias()`.

Acoplamento a observar. A aba de experimento lê os endereços VISA diretamente do `QSettings`, e não por sinal vindo do painel de ligações. É um canal lateral: funciona, mas cria uma dependência invisível entre os dois componentes. Endereços de instrumentos simulados nunca são gravados nessas chaves, para que desligar o modo demonstração devolva o último instrumento real.

3 Camada de hardware

3.1 O driver da fonte

`PWS4323_Driver` encapsula os comandos SCPI da fonte:

Método	Comando SCPI	Efeito
<code>configure_safety_limits</code>	<code>SOUR:CURR</code>	Define o limite de corrente
<code>set_output</code>	<code>OUTP ON/OFF</code>	Liga ou desliga a saída
<code>set_voltage</code>	<code>SOUR:VOLT</code>	Programa a tensão
<code>measure_current</code>	<code>MEAS:CURR?</code>	Lê a corrente entregue
<code>turn_off_safely</code>	<code>SOUR:VOLT 0 → OUTP OFF</code>	Zera e desliga

`turn_off_safely` zera a tensão *antes* de desligar a saída, e engole exceções: é chamado em caminhos de erro, onde falhar ruidosamente deixaria a fonte energizada.

3.2 O driver do multímetro

DMM4050_Driver é mais sensível à ordem das operações. Na abertura:

```
if "SOCKET" in resource_string.upper():
    self.inst.write_termination = '\n' # conexões por rede exigem
    self.inst.read_termination = '\n' # terminação explícita
self.inst.write("SYST:REM") # 1. força modo remoto
self.inst.write("*CLS") # 2. limpa o buffer de erros
```

Na configuração da medida:

```
self.inst.write("CONF:CURR:DC")
self.inst.write("SENS:CURR:DC:RANG 1e-4") # faixa FIXA de 100 uA
self.inst.write(f"SENS:CURR:DC:NPLC {nplc}")
```

A faixa fixa de 100 μA é essencial. Com auto-range, o instrumento escolhe uma escala maior e os nanoampères da fotocorrente desaparecem na resolução. O NPLC 10 integra cada leitura por dez ciclos da rede, o que filtra ruído de 50/60 Hz ao custo de tempo de aquisição.

Ao fechar, o driver envia SYST:LOC antes de encerrar a comunicação, devolvendo o controle ao painel frontal do aparelho.

3.3 Threads de infraestrutura

Duas operações de rede não podem bloquear a interface, e por isso vivem em threads próprias:

ScannerThread

Lista os recursos VISA disponíveis, filtra a fonte pelo identificador de fabricante no endereço USB, e testa o endereço de rede do multímetro com um *IDN? de timeout curto. Emite duas listas de pares (nome amigável, endereço).

ValidatorThread

Abre um recurso, pergunta *IDN? e emite o resultado. Endereços que começam com DEMO:: respondem sem passar por VISA.

HardwareManager é o QObject que orquestra as duas e retransmite seus sinais para a interface. Ele mantém referências às threads em execução — sem isso, o coletor de lixo do Python destruiria um QThread ainda rodando — e as solta quando cada thread termina, usando o sinal finished e um wait() de confirmação. Uma varredura em curso bloqueia novas até terminar.

3.4 Seleção de drivers e modo demonstração

Um único ponto decide qual par de drivers usar:

```
def obter_drivers(modos_demonstracao: bool) -> tuple:
    if modos_demonstracao:
        return MockPWS4323_Driver, MockDMM4050_Driver
    return PWS4323_Driver, DMM4050_Driver
```

Como os mocks expõem exatamente a mesma interface, quem consome não precisa saber qual recebeu: instancia e usa. O ExperimentWorker só faz uma distinção adicional — não abre o ResourceManager do PyVISA quando está em modo demonstração.

4 O modelo físico

Todo o cálculo vive em utils/math_models.py, em funções puras que recebem e devolvem vetores numpy. As constantes usadas são as da CODATA:

Símbolo	Valor	Grandeza
c	299 792 458 m/s	Velocidade da luz
k_B	$1,380649 \times 10^{-23}$ J/K	Constante de Boltzmann
h_{ref}	$6,62607015 \times 10^{-34}$ J·s	Constante de Planck

4.1 Da resistência à temperatura

A resistência do tungstênio varia com a temperatura segundo um polinômio de segundo grau, com T_c em graus Celsius:

$$R(T_c) = R_0 (1 + \alpha T_c + \beta T_c^2) \quad (1)$$

R_0 é a resistência a 0 °C, não a medida a frio

A equação (1) define R_0 como a resistência em $T_c = 0$ °C. O operador, porém, mede o filamento frio na temperatura *ambiente*, onde a resistência já é cerca de 13% maior. Usar a medida diretamente como R_0 envia todas as temperaturas — e portanto h .

A conversão é a própria equação (1) invertida na temperatura em que a medida foi feita:

$$R_0 = \frac{R(T_{\text{amb}})}{1 + \alpha T_{\text{amb}} + \beta T_{\text{amb}}^2} \quad (2)$$

É o que faz `corrigir_r0_para_zero_celsius`. A interface pede os dois valores — a resistência medida e a temperatura em que foi medida — e exibe ao vivo o R_0 resultante, que é o valor efetivamente usado no cálculo. Para 1,2 Ω medidos a 25 °C, $R_0 = 1,0608 \Omega$.

Invertendo o modelo

O experimento mede R e precisa de T . Reorganizando a equação (1) como uma equação do segundo grau em T_c :

$$\underbrace{R_0 \beta}_{a} T_c^2 + \underbrace{R_0 \alpha}_{b} T_c + \underbrace{(R_0 - R)}_c = 0 \quad (3)$$

e resolvendo por Bhaskara, tomando a raiz positiva (a única com sentido físico, pois R cresce com T):

$$T_c = \frac{-R_0 \alpha + \sqrt{(R_0 \alpha)^2 - 4 R_0 \beta (R_0 - R)}}{2 R_0 \beta}, \quad T[\text{K}] = T_c + 273,15 \quad (4)$$

É exatamente isso que `calculate_temperature` implementa, de forma vetorizada.

Onde a inversão engana. Quando a resistência medida fica *abaixo* de R_0 — o que acontece em tensões próximas de zero — a equação (4) ainda tem solução matemática, mas ela não tem sentido físico. É a origem das temperaturas da ordem de 77 K que aparecem no início das varreduras completas. Em casos extremos (R negativa) chega a produzir kelvin negativo. O discriminante só fica negativo para resistências muito abaixo de R_0 ; nesse caso a função devolve NaN, ponto a ponto, em vez de lixo. Mas o caso comum — a raiz real e absurda — não é barrado aqui: quem o descarta é a seleção de pontos da regressão (seção 4.4). A separação é proposital: esta função inverte o modelo, aquela decide o que é utilizável.

4.2 Da temperatura à constante de Planck

A distribuição de Planck para a radiância espectral de um corpo negro é:

$$I(\lambda, T) = \frac{2\pi c^2 h}{\lambda^5} \frac{1}{e^{hc/\lambda k_B T} - 1} \quad (5)$$

Quando $hc/(\lambda k_B T) \gg 1$, o termo -1 do denominador é desprezível e vale a **aproximação de Wien**:

$$I(\lambda, T) \approx \frac{2\pi c^2 h}{\lambda^5} \exp\left(-\frac{hc}{\lambda k_B T}\right) \quad (6)$$

Nas condições deste experimento ($\lambda = 590$ nm, T até 3000 K) o expoente vale cerca de 8, e o erro cometido pela aproximação é da ordem de $e^{-8} \approx 0,03\%$ — desprezível diante das demais fontes de erro.

O LED responde linearmente à radiação que recebe na sua faixa espectral, então a fotocorrente medida é proporcional a $I(\lambda, T)$. Tomando o logaritmo da equação (6):

$$\ln I = \underbrace{\ln\left(\frac{2\pi c^2 h}{\lambda^5}\right)}_{\text{constante}} - \frac{hc}{\lambda k_B} \cdot \frac{1}{T} \quad (7)$$

Ou seja: $\ln I$ contra $1/T$ é uma reta, cuja inclinação é

$$m = -\frac{hc}{\lambda k_B} \quad \Rightarrow \quad \boxed{h = -\frac{m \lambda k_B}{c}} \quad (8)$$

Toda a montagem experimental existe para medir essa inclinação.

4.3 A regressão

`calculate_planck_constant` executa a cadeia completa:

1. **Filtra os pontos utilizáveis.** Descarta temperaturas nulas ou não finitas (que estourariam o cálculo de $1/T$) e fotocorrentes abaixo de um limiar de confiança.
2. **Lineariza.** Constrói $x = 1/T$ e $y = \ln I$.
3. **Ajusta por mínimos quadrados.** Resolve $y = mx + c$ com `numpy.linalg.lstsq`. Exige ao menos dois pontos válidos; abaixo disso devolve zeros.
4. **Avalia a qualidade.** Calcula $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$, com guarda para o caso $\text{SS}_{\text{tot}} = 0$ (todos os $\ln I$ iguais), em que R^2 é indefinido e se reporta zero.
5. **Extraí h** pela equação (8) e calcula o erro relativo contra h_{ref} .

Retorna a tupla `(h, erro_relativo, m, c, r2)`.

4.4 O limiar de confiança e a região de Wien

O passo 1 é o mais delicado do software, e merece detalhamento.

A fotocorrente cai exponencialmente com a queda da temperatura. Abaixo de aproximadamente 1800 K ela se torna menor que o ruído de fundo do multímetro: o que se lê não é mais sinal, é o piso do instrumento. Esses pontos não carregam informação sobre h , mas entram na regressão como se carregassem, e a distorcem.

A seleção é feita por uma função dedicada, `selecionar_pontos_validos`, que aplica três critérios e devolve uma máscara booleana:

1. **Temperatura fisicamente possível** — descarta NaN e qualquer $T \leq 0$;
2. **Região de Wien** — descarta T abaixo de um limite configurável, por padrão 1800 K;
3. **Fotocorrente acima do limiar** — mantém o logaritmo finito.

A mesma função alimenta a interface, que pinta de cinza os pontos descartados: o operador vê, ponto a ponto, o que entrou e o que não entrou na conta.

```
temperatura_utilizavel = np.isfinite(T_kelvin) & (T_kelvin > 0)
regiao_de_wien        = T_kelvin >= t_minima
sinal_acima_do_piso    = np.isfinite(photocurrent) & (photocurrent >
    limiar_corrente)
return temperatura_utilizavel & regiao_de_wien & sinal_acima_do_piso
```

O efeito do critério pode ser medido. Numa varredura completa de 0 a 12 V com 49 pontos, gerada pela bancada simulada:

Critério de corte	Pontos	h (J·s)	Erro	R^2
Nenhum	49	$6,9 \times 10^{-36}$	99,0%	0,062
Só $I > 1$ nA	49	$6,9 \times 10^{-36}$	99,0%	0,062
$I > 25$ nA	27	$6,32 \times 10^{-34}$	4,6%	0,999
$T > 1800$ K (padrão)	29	$6,31 \times 10^{-34}$	4,8%	0,999
$T > 2000$ K	23	$6,42 \times 10^{-34}$	3,1%	0,999

Leitura do quadro. Um limiar de 1 nA sobre a corrente *não descarta nenhum ponto*: o próprio piso do multímetro é da ordem de 4 nA, portanto maior que o limiar. É por isso que o corte útil é o de *temperatura*, e não o de corrente. Um limiar de corrente coerente com o instrumento precisaria vir do modelo de erro dele — a especificação de exatidão na faixa de 100 μ A tem um termo de fundo de 25 nA — e não de um número escolhido à mão; essa derivação é trabalho ainda planejado.

O ponto central de operação. O corte acontece na *regressão*, não na *coleta*. A varredura pode cobrir a faixa inteira, inclusive a partir de 0 V, e o CSV guarda todos os pontos; apenas a conta final usa a região informativa. Não é preciso escolher entre registrar o comportamento completo do filamento e obter um bom valor de h .

4.5 A simulação analítica

`simulate_experiment_data` gera dados sintéticos para a página de Simulação com a física do experimento real. Cada tensão do vetor de varredura passa pelo balanço de energia em regime estacionário,

$$\frac{V^2}{R(T)} = k_{\text{rad}} (T^4 - T_{\text{amb}}^4) + k_{\text{cond}} (T - T_{\text{amb}}), \quad (9)$$

resolvido por bissecção (`resolver_temperatura_regime`). A razão $k_{\text{cond}}/k_{\text{rad}}$ vem do ajuste sobre os 1127 pontos úteis dos CSVs reais, e a escala absoluta é calibrada para o filamento em uso bater a âncora medida na bancada: 2540 K a 12 V (`calibrar_dissipacao`). Tudo o mais deriva daí: $R(T)$ pela equação (1), $i = V/R$, e a fotocorrente pela equação (6) com h_{ref} embutido — recuperá-lo pela regressão é o teste de coerência da cadeia inteira.

Dois artefatos do instrumento real são reproduzidos de propósito: o fundo de $\sim 4,5$ nA que o DMM4050 lê em tensão baixa (para a simulação exibir os mesmos pontos frios sem sinal que a seleção da região de Wien existe para cortar) e um ruído gaussiano proporcional ao sinal, com fator escolhido pelo operador. Um parâmetro de semente congela o gerador para testes reprodutíveis.

História. Até a Fase 7 esta função impunha $T = \text{linspace}(1500, 3000)$ *independente das tensões* — a relação $V \rightarrow T$, que é o coração do experimento, não existia na simulação (defeito A9 da análise inicial). Hoje a página de Simulação e a bancada simulada (seção 6) resolvem a *mesma* equação (9), pelo mesmo código em `utils/math_models.py`, calibrado nos mesmos dados reais. A diferença entre as duas é de propósito, não de física: a simulação é vetorizada e instantânea; a bancada simulada emula os instrumentos um a um (erros de setting/readback, quantização do DMM) para exercitar o mesmo caminho de código da coleta real.

5 Teoria de erros

Esta seção está sujeita a revisão acadêmica. As escolhas de modelagem abaixo estão explicitadas justamente para poderem ser contestadas. A lista de pontos a verificar está em `PENDENCIAS.txt`, item P2. Implementação em `utils/error_models.py`.

5.1 Vocabulário (GUM)

Incerteza Tipo A

Estimada por meios estatísticos, a partir da repetição da medida. Para N leituras de desvio padrão amostral s :

$$u_A = \frac{s}{\sqrt{N}} \quad (10)$$

Incerteza Tipo B

Estimada por qualquer outro meio — aqui, pela especificação do fabricante. Um datasheet que promete $\pm(a\% \text{ da leitura} + b \text{ da faixa})$ está declarando um *limite*, não um desvio padrão. Sem informação sobre a forma da distribuição dentro desse limite, o GUM manda assumir distribuição retangular:

$$u_B = \frac{a}{\sqrt{3}} \quad (11)$$

Combinação

Contribuições independentes somam em quadratura:

$$u = \sqrt{u_A^2 + u_B^2} \quad (12)$$

Propagação

Para $f(x, y, \dots)$ com variáveis independentes:

$$u_f^2 = \left(\frac{\partial f}{\partial x}\right)^2 u_x^2 + \left(\frac{\partial f}{\partial y}\right)^2 u_y^2 + \dots \quad (13)$$

Incerteza expandida

Para declarar um intervalo de confiança:

$$U = k \cdot u_c \quad (k = 2 \Rightarrow \approx 95\%) \quad (14)$$

5.2 Aleatório versus sistemático: a distinção que organiza tudo

Este é o ponto mais importante da seção, e o mais fácil de errar.

Nem toda incerteza pode entrar no peso de um ponto individual. Considere R_0 : ele é o *mesmo* para toda a varredura. Erro R_0 desloca todas as temperaturas na mesma direção — não espalha os pontos, translada a nuvem inteira. Colocá-lo no peso individual equivale a dizer ao ajuste que cada ponto flutua sozinho tanto quanto o conjunto se desloca junto. O efeito prático é inflar os pesos e derrubar o χ^2 reduzido artificialmente.

A implementação separa, portanto:

	Aleatórias	Sistemáticas
Quais são	Leitura de V e i ; fotocorrente (datasheet e Tipo A)	$R_0, \alpha, \beta, \lambda$
Comportamento	Variam ponto a ponto	Comuns a toda a varredura
Como entram	No peso de cada ponto do ajuste	Propagadas ao final, sobre o resultado

Para cada sistemática, o software refaz o ajuste com o parâmetro deslocado de $+u$ — mantendo a *mesma* seleção de pontos, pois se quer a sensibilidade ao parâmetro e não à seleção — e mede o quanto h se move:

$$u_{h,\text{sist}}^{(p)} = |h(p + u_p) - h(p)| \quad (15)$$

É o mesmo espírito de $(\partial f / \partial p) u_p$ da equação (13), obtido numericamente porque a dependência de h em R_0 passa por dentro do ajuste e não tem forma fechada simples.

5.3 A cadeia completa

Passo	O que acontece
1	$u(V), u(i)$ das especificações de readback da fonte
2	$R = V/i - R_{\text{cabos}}$, com u_R
3	T pela Bhaskara, com u_T aleatória (só de u_R)
4	$u(I_{\text{LED}})$ do datasheet, combinada com o Tipo A
5	Seleção dos pontos (região de Wien e limiar do instrumento)
6	$x = 1/T, y = \ln I$, com suas incertezas
7	Ajuste ponderado com erro nos dois eixos $\rightarrow m \pm u_m$
8	$h = -m\lambda k_B / c$
9	Sensibilidade às sistemáticas R_0, α, β
10	Combinação final e $U = k u_h$

Passo 2 — resistência

Para um quociente, as incertezas *relativas* somam em quadratura:

$$\left(\frac{u_{V/i}}{V/i}\right)^2 = \left(\frac{u_V}{V}\right)^2 + \left(\frac{u_i}{i}\right)^2, \quad u_R^2 = u_{V/i}^2 + u_{R_{\text{cabos}}}^2 \quad (16)$$

Passo 3 — temperatura

Derivando implicitamente $R = R_0(1 + \alpha T_c + \beta T_c^2)$ e chamando $D = R_0(\alpha + 2\beta T_c)$, que é dR/dT_c :

$$\frac{\partial T}{\partial R} = \frac{1}{D} \quad (17)$$

$$\frac{\partial T}{\partial R_0} = -\frac{1 + \alpha T_c + \beta T_c^2}{D} \quad (18)$$

$$\frac{\partial T}{\partial \alpha} = -\frac{R_0 T_c}{D} \quad (19)$$

$$\frac{\partial T}{\partial \beta} = -\frac{R_0 T_c^2}{D} \quad (20)$$

As quatro são conferidas contra derivação numérica nos testes, com concordância em seis dígitos. Quando $D \rightarrow 0$ o modelo perde sensibilidade — a curva $R(T)$ fica horizontal e a inversão deixa de ser bem posta — e a implementação devolve infinito, para que o ponto apareça com incerteza enorme em vez de passar despercebido.

Passos 6 e 7 — linearização e ajuste

$$u_x = \frac{u_T}{T^2} \quad u_y = \frac{u_I}{I} \quad (21)$$

Com peso $w_i = 1/u_{y_i}^2$, minimizar $\chi^2 = \sum w_i (y_i - mx_i - c)^2$ leva às somas $S = \sum w$, $S_x = \sum wx$, $S_y = \sum wy$, $S_{xx} = \sum wx^2$, $S_{xy} = \sum wxy$, e:

$$\Delta = SS_{xx} - S_x^2, \quad m = \frac{SS_{xy} - S_x S_y}{\Delta}, \quad u_m = \sqrt{\frac{S}{\Delta}} \quad (22)$$

Como u_x não é desprezível, usa-se o método da **variância efetiva** (Orear 1982; equivalente ao de York 1968 para retas), que projeta o erro de x sobre y pela própria inclinação:

$$w_i = \frac{1}{u_{y_i}^2 + m^2 u_{x_i}^2} \quad (23)$$

O peso depende de m , então itera-se: parte-se do m do ajuste simples, recalculam-se os pesos, reajusta-se, até m estabilizar. Converge em poucas iterações.

Por que não uma rotina pronta de ODR. A implementação é própria, em vinte linhas de `numpy`, por três razões: `scipy.odr` está depreciada e será removida na versão 1.19; o método cabe inteiro numa página e pode ser auditado; e não acrescenta dependência ao projeto. Se o orientador exigir ODR completa, a troca é localizada numa única função.

Passo 10 — combinação

$$u_h^2 = \underbrace{\left(\frac{u_m}{m}\right)^2}_{\text{ajuste}} h^2 + \underbrace{\left(\frac{u_\lambda}{\lambda}\right)^2}_{\lambda} h^2 + \sum_p (u_{h,\text{sist}}^{(p)})^2 \quad (24)$$

com $u_\lambda = \Delta\lambda/\sqrt{3}$ e $U = k u_h$, $k = 2$.

5.4 Como ler o χ^2 reduzido neste experimento

O χ^2 reduzido responde "as incertezas declaradas explicam a dispersão observada?". Valor perto de 1 significa que sim.

Aqui ele fica sistematicamente abaixo de 1, e isso é esperado. A incerteza da fotocorrente vem de uma especificação de datasheet, que é um *limite de pior caso* tratado como distribuição retangular. O ruído real do instrumento é bem menor que o pior caso garantido pelo fabricante. Declarar o limite e observar dispersão menor que ele produz χ^2_{red} pequeno por construção — aproximadamente o quadrado da razão entre o ruído real e a incerteza declarada.

O χ^2 só tenderia a 1 se a incerteza declarada fosse o desvio padrão verdadeiro do processo aleatório. Ele continua útil como diagnóstico *relativo*: uma mudança brusca entre coletas indica que algo mudou na montagem.

5.5 O orçamento de incertezas

O software reporta a fatia de cada fonte na variância de h . Num caso típico da bancada simulada, varrendo de 0 a 12 V:

Fonte	Contribuição
λ (largura espectral do LED)	53%
Inclinação (aleatório do ajuste)	41%
R_0 (sistemático)	6%

A largura espectral do LED domina, como previsto: são dezenas de nanômetros sobre um λ de 590 nm. Nenhuma melhoria na eletrônica reduz esse termo — só um sensor mais seletivo reduziria.

5.6 O que NÃO está no orçamento

Erros sistemáticos de natureza física não entram: emissividade do tungstênio (o filamento é corpo cinza, não negro), reflexões na montagem, alinhamento entre LED e filamento, e gradiente de temperatura ao longo do filamento. Se o valor da CODATA cair fora da incerteza declarada, é aí que se deve procurar — e o software diz explicitamente quando isso acontece.

6 A bancada simulada

`core/mock_hardware.py` implementa um filamento e um LED virtuais que permitem exercitar todo o software sem instrumentos.

6.1 Por que os dois drivers compartilham estado

Na bancada real, fonte e multímetro estão acoplados pelo filamento: a tensão que a fonte aplica define a temperatura, e é a temperatura que o LED enxerga. Dois mocks independentes não conseguiriam reproduzir isso. Por isso ambos apontam para uma instância única de `BancadaSimulada`, que guarda o estado físico.

6.2 O modelo térmico

Em regime estacionário, a potência elétrica entregue ao filamento iguala a potência dissipada:

$$\frac{V^2}{R(T)} = k_{\text{rad}}(T^4 - T_{\text{amb}}^4) + k_{\text{cond}}(T - T_{\text{amb}}) \quad (25)$$

O primeiro termo é a radiação (lei de Stefan–Boltzmann); o segundo agrupa as perdas por condução e convecção, que dominam em temperaturas baixas. Com $R(T)$ dado pela equação (1),

a diferença entre os dois lados é monótona decrescente em T , o que permite resolver por bissecção sem precisar de derivada:

```
def desequilibrio(t):
    return tensao**2 / self.resistencia(t) - (
        self.k_rad * (t**4 - T_AMBIENTE**4)
        + self.k_cond * (t - T_AMBIENTE))
```

6.3 Como as constantes foram obtidas

k_{rad} e k_{cond} não foram arbitrados. Foram ajustados por mínimos quadrados sobre os 1127 pontos úteis dos 52 arquivos CSV de coletas reais guardados em `data_backup/`, resultando em resíduo mediano de 2,9% na potência. A 2500 K, o termo radiativo responde por 99% da dissipação — coerente com a física esperada.

Do ajuste preserva-se a *razão* $k_{\text{cond}}/k_{\text{rad}}$, que descreve a forma da curva. A escala é recalibrada na construção do objeto, por bissecção, para que o filamento virtual atinja 2540 K em 12 V — o topo efetivamente medido na bancada real.

A constante do LED é fixada de modo que a fotocorrente na âncora valha $7,2 \times 10^{-7}$ A, o valor máximo observado nas coletas reais. O expoente usa h_{ref} : os dados gerados *contêm* o valor verdadeiro de h , e recuperá-lo é o teste de que a cadeia de cálculo está correta.

6.4 Erros e ruídos reproduzidos

Efeito	Como é modelado
Erro de programação da fonte	Distribuição retangular sobre o valor pedido; o valor efetivamente aplicado difere do programado.
Erro de leitura da fonte	Distribuição retangular, com termos proporcional e fixo, separadamente para tensão e corrente.
Resolução do multímetro	Leitura arredondada para passos de 100 pA.
Piso do multímetro	Um deslocamento sistemático de alguns nanoampères mais ruído gaussiano — reproduz o que os CSVs reais mostram em tensão baixa.
Modo corrente constante	Se a corrente calculada excede o limite configurado, a fonte segura a corrente e a tensão nos terminais cai.
Saída desligada	Corrente nula e temperatura de volta ao ambiente.

O gerador aleatório aceita semente, o que torna uma varredura inteiramente reprodutível — útil para testes de regressão.

O piso é reproduzido de propósito. Sem ele, o mock esconderia justamente o problema descrito na seção 4.4, e não haveria como verificar a correção quando ela for implementada.

6.5 O que o mock ainda não reproduz

Inércia térmica — o filamento virtual salta direto ao regime estacionário, o que é aceitável porque os workers já esperam o tempo de estabilização — e a resistência dos cabos da medição a dois fios.

7 Concorrência

7.1 Por que threads

Uma coleta real leva minutos e é dominada por esperas: estabilização térmica, ida e volta de cada comando SCPI. Fazer isso na thread da interface congelaria a janela. Cada coleta roda, portanto, em um `QThread` próprio, que se comunica com a interface exclusivamente por sinais.

Os drivers VISA são instanciados *dentro* da thread que os usa. Isso é deliberado: conexões VISA não devem ser compartilhadas entre threads.

7.2 Os sinais

Sinal	Carga	Quando
<code>new_data_point</code>	tensão, temperatura, fotocorrente	A cada ponto medido
<code>finished_exp</code>	h , erro, m , c , R^2	Ao concluir a regressão
<code>error_occurred</code>	mensagem	Em falha de comunicação ou parada precoce

7.3 Parada segura

`stop()` apenas baixa uma flag; a thread a consulta no topo de cada iteração e sai do laço por conta própria. A interface *não* espera pela thread nesse momento — ela continua respondendo, e a thread desliga a fonte, calcula o resultado parcial e emite `finished_exp` naturalmente.

O bloco `finally` do laço de coleta é o que garante a segurança da bancada: aconteça o que acontecer, a fonte é zerada, desligada e fechada, e o multímetro devolvido ao modo local.

```
finally:
    if pws:
        pws.turn_off_safely()    # zera a tensão e desliga a saída
        pws.close()
    if dmm:
        dmm.close()             # devolve o painel frontal ao operador
```

8 O fluxo de um experimento

Passo a passo, do clique ao resultado:

1. A interface lê os nove campos de parâmetros, converte para número e monta um dicionário.
2. Lê do `QSettings` o modo demonstração e os endereços dos instrumentos.
3. Dimensiona a barra de progresso pelo *mesmo* vetor de tensões que o worker vai percorrer — calcular o total por outra fórmula produz divergências por aritmética de ponto flutuante.
4. Cria o `ExperimentWorker`, conecta os três sinais e inicia a thread.
5. O worker escolhe o par de drivers, instancia ambos, configura o limite de corrente da fonte e a faixa/integração do multímetro.
6. Escreve o cabeçalho do CSV.
7. Liga a saída da fonte e entra no laço. Para cada tensão:

- a. programa a tensão;
 - b. aguarda o tempo de estabilização térmica;
 - c. lê a corrente na fonte e a fotocorrente no multímetro;
 - d. calcula $R = V/i$, com piso na corrente para evitar divisão por zero;
 - e. converte R em T pela equação (4);
 - f. grava a linha no CSV *imediatamente*, com abertura e fechamento do arquivo a cada ponto;
 - g. emite `new_data_point`, e a interface atualiza gráficos, registro e barra.
8. Ao fim (ou na parada), desliga a saída e, havendo mais de dois pontos, chama `calculate_planck_constant` e emite o resultado.
 9. O bloco `finally` garante o desligamento seguro.

Sobre a resistência calculada. A resistência do filamento vem da tensão *lida de volta* da fonte (MEAS:VOLT?), não do valor programado: o readback do PWS4323 é mais exato que a exatidão de programação. Se o instrumento não responder ao readback, o cálculo cai para o valor programado em vez de abortar a coleta.

Como a medição é a dois fios, a resistência dos cabos entra somada à do filamento; a interface oferece um campo para descontá-la:

$$R_{\text{filamento}} = \frac{V_{\text{medida}}}{i} - R_{\text{cabos}}$$

As duas tensões — programada e medida — são gravadas no CSV, o que permite auditar depois a diferença entre elas.

9 Formato dos dados

9.1 O CSV de coleta

Um arquivo por execução, em `Software/data_backup/`:

Coletas reais	<code>exp_planck_AAAAMMDD_HHMMSS.csv</code>
Coletas simuladas	<code>demo_planck_AAAAMMDD_HHMMSS.csv</code>

Prefixos distintos impedem que dados de demonstração contaminem o acervo de medidas reais — que é, entre outras coisas, a base de calibração da bancada simulada.

Coluna	Unidade	Origem
Tensao_Fonte_V	V	Valor programado na fonte
Corrente_Filamento_A	A	MEAS:CURR? na fonte
Resistencia_Ohms	Ω	Calculada, $V_{\text{medida}}/i - R_{\text{cabos}}$
Temperatura_K	K	Calculada, equação (4)
Fotocorrente_A	A	READ? no multímetro
Tensao_Medida_V	V	MEAS:VOLT? na fonte

Há dados históricos gravados neste formato. A coluna de tensão medida foi acrescentada *ao final*, justamente para que as cinco colunas originais permaneçam na mesma ordem: arquivos antigos continuam legíveis, e leitores que acessam por posição não quebram. Qualquer alteração futura de esquema deve seguir a mesma regra.

9.2 O relatório PDF

`generate_planck_report` monta o documento com `reportlab`, recebendo quatro blocos: resumo de resultados, dicionário de parâmetros, metadados informados pelo usuário e o caminho de uma imagem. A imagem é uma captura da área de gráficos, gerada no momento da exportação e apagada em seguida.

10 Perfis e rastreabilidade

10.1 O sistema de perfis

Quatro famílias de parâmetros vivem em JSONs editáveis dentro de `Software/profiles/`:

Arquivo	Conteúdo
<code>leds.json</code>	$\lambda \pm \Delta\lambda$ de cada sensor
<code>filamentos.json</code>	coeficientes α e β do tungstênio
<code>instrumentos.json</code>	especificações de exatidão, que alimentam a teoria de erros
<code>varreduras.json</code>	presets de faixa, passo, espera e leituras por ponto

Três decisões governam o sistema:

1. **Todo perfil de constante física carrega a sua fonte.** Um número sem procedência é uma incerteza sistemática invisível. O campo é obrigatório, e quando a origem é desconhecida isso fica escrito no próprio perfil. Perfis de varredura são exceção: são escolha operacional do experimentador, não valor a ser citado.
2. **O JSON manda, mas nunca derruba o software.** Arquivo ausente, corrompido ou com campos faltando faz o carregador cair para os padrões embutidos em `content/perfis.py` e registrar um aviso, exibido no painel de parâmetros. Um laboratório não pode ficar sem coletar porque alguém errou uma vírgula.
3. **R_0 não é perfil.** A resistência a frio é propriedade do filamento específico que está na bancada, medida a cada montagem — não de um modelo tabelado. Continua sendo campo preenchido por experimento.

Trocar de multímetro passa a ser editar `instrumentos.json`: as especificações são convertidas em `EspecificacaoInstrumento` e entram direto na cadeia de incertezas descrita na seção 5.

10.2 O painel de parâmetros compartilhado

Antes, os mesmos campos existiam duplicados nas abas de Simulação e de Bancada. Mudar um rótulo, um padrão ou uma unidade exigia lembrar dos dois lugares — e eles foram divergindo.

Agora há um componente só, `ui/components/painel_parametros.py`, que cada aba instancia no seu modo:

Modo	O que muda
<code>bancada</code>	inclui resistência de cabos e leituras por ponto
<code>simulacao</code>	inclui fator de ruído, sem os campos de hardware

O método `coletar()` devolve o dicionário de parâmetros já com R_0 corrigido e sua incerteza propagada, e é o único ponto onde os campos viram números.

10.3 Metadados por coleta

Cada coleta produz dois arquivos irmãos:

```
data_backup/exp_planck_AAAAMDD_HHMMSS.csv - as medidas
data_backup/exp_planck_AAAAMDD_HHMMSS.json - como foram obtidas
```

O JSON registra os perfis usados, a resistência medida a frio *com a temperatura em que foi medida*, o R_0 corrigido, os coeficientes, $\lambda \pm \Delta\lambda$, todos os parâmetros de varredura, o modo (bancada real ou demonstração), o ambiente de execução e, ao final, o resultado completo com incerteza, diagnósticos do ajuste e orçamento.

É gravado **duas vezes**: na abertura da coleta — para que os parâmetros sobrevivam a uma queda de energia no meio — e no encerramento, acrescentando os resultados.

Por que isto importa. O CSV sempre guardou as medidas, mas não com *quais parâmetros* foram processadas. O custo disso foi concreto: para calibrar a bancada simulada foi preciso *deduzir*, a partir dos próprios números, que as coletas históricas haviam usado $R_0 \approx 0,42 \Omega$. Os 52 CSVs antigos seguem sem metadados — para eles aquele valor continua sendo uma dedução, não um registro.

11 Camada de conteúdo

`content/referencias.py` guarda a lista de referências como dados tipados, não como código de interface. Cada item é uma `dataclass Referencia` congelada, com título, autoria, publicação, identificador, nome do arquivo local, URLs oficiais, condição de acesso, o uso que o software faz do documento e a lista de trechos aproveitados.

Duas propriedades calculadas resolvem o caminho do arquivo na pasta `Docs/` e informam se ele está presente — a interface usa isso para desabilitar o botão de cópia local quando o PDF não existe.

Para acrescentar um documento, basta acrescentar um item à lista: a aba se adapta sozinha.

12 A interface Fluent

12.1 O que mudou e por quê

A interface anterior tinha quatro problemas concretos de navegação, todos resolvidos na migração:

Problema	Solução
Abas dentro de abas	Parâmetros ganharam página própria; as páginas de coleta só executam e mostram
Estado do hardware invisível fora da primeira aba	Cabeçalho fixo com os dois instrumentos, acima da área de páginas
Parâmetros repetidos entre Simulação e Bancada	Uma página só; as duas coletas leem o mesmo dicionário
Sem retorno do que está acontecendo	Barra de status permanente com ponto, temperatura e arquivo em gravação

12.2 Estrutura

`JanelaPlanck` estende `FluentWindow` e registra cinco sub-interfaces. Cada página é um `QWidget` com `objectName` definido — exigência do `addSubInterface`.

As duas páginas de coleta partilham `PaginaExecucaoBase`, que concentra gráficos, barra de progresso, registro em tempo real, cartão de resultado e exportação de PDF. As subclasses só definem como a coleta começa: a de Simulação instancia `SimulationWorker`, a de Bancada verifica o modo, resolve os recursos VISA e instancia `ExperimentWorker`.

Os workers não mudaram. `SimulationWorker` e `ExperimentWorker` são exatamente os mesmos das fases anteriores. Foi possível porque já se comunicavam só por sinais, sem conhecer a interface — a separação de camadas pagou-se aqui.

12.3 Configuração que saiu do código

Dois valores que estavam fixos no código viraram campos persistidos:

Item	Antes	Agora
Limite de corrente	2,0 A fixo, com comentário dizendo 1,5 A	Campo de 0,1 a 3,0 A, padrão 1,5 A, gravado nos metadados
Endereço do multímetro	escrito no código	Campos de IP e porta; <code>endereco_dmm()</code> monta a string VISA

12.4 Transição

A janela antiga continua funcional e é escolhida por variável de ambiente:

```
set PLANCK_UI=classica && python main.py
```

Isso não é permanente. A janela antiga — e com ela `ui/theme.py` e o painel de ligações original — sai quando a nova tiver rodado um experimento de verdade com os instrumentos ligados.

Licença. A versão comunitária do `QFluentWidgets` é GPLv3. Enquanto o software rodar apenas internamente não há obrigação, porque não há distribuição; a obrigação nasce ao distribuir executáveis, que é o que a fase de empacotamento prevê. Ver `PENDENCIAS.txt`, item P6.

13 Aparência

`ui/theme.py` exporta uma única string QSS aplicada globalmente. A paleta é escura, com fundo quase preto, superfícies em cinza-escuro e azul como cor de destaque. Os gráficos do `pyqtgraph` são configurados separadamente, com fundo e primeiro plano combinando com o tema.

Alguns componentes ainda trazem estilo embutido no próprio código, em vez de virem da folha global. É um ponto de dívida técnica conhecido, a resolver numa revisão de interface.

14 Como estender

Acrescentar uma referência

Adicione um item à lista em `content/referencias.py`, com o nome do arquivo dentro de `Docs/`. Nada mais precisa mudar.

Suportar outro instrumento

Escreva uma classe com os mesmos métodos públicos do driver existente e faça `obter_drivers` devolvê-la. Nenhum outro arquivo precisa saber. O arquivo de testes verifica automaticamente que a interface do mock cobre a do driver real — o mesmo critério vale para um driver novo.

Alterar o modelo físico

Todas as contas estão em `utils/math_models.py`, em funções puras sem dependência de Qt. São testáveis isoladamente.

Acrescentar uma aba

Crie o widget em `ui/tabs/` e registre-o em `setup_ui`, na janela principal.

15 Testes

`Tests/test_mock_hardware.py` roda sem hardware e sem dependências extras:

```
cd Software && python ../Tests/test_mock_hardware.py
```

Cobre quatro grupos de verificação: que os mocks são substituíveis pelos drivers reais (comparando as interfaces por introspecção), que a física simulada está calibrada nos pontos de âncora e é monótona, que os comportamentos do instrumento aparecem (piso do multímetro, modo corrente constante, corte seguro), e que a cadeia de cálculo recupera h dos dados gerados.

Um dos testes fixa deliberadamente o *erro esperado* numa varredura completa. Quando o critério de corte descrito na seção 4.4 for corrigido, esse teste falhará — e é assim que se saberá que a correção funcionou.